# Lab 3 questions and code locator

Explained answers, file names, source lines, and code excerpts for your demonstration.

Prepared October 13, 2026. Line numbers describe the current source snapshot; search for the expression if later edits move the code.

# Instructor questions: files, lines, and code to show

Line numbers were captured from the saved source on October 13, 2026. They are a navigation aid, not something to memorize. Edits can move them; use the method or code expression below as the reliable search anchor. All application paths start inside app-crud unless labelled workspace root.

## How to locate the answer during a demonstration

1. In VS Code press Ctrl+P and enter the listed file path.
2. Press Ctrl+G, enter the start line number, and press Enter.
3. Point to the expression, name what it checks/does, and explain the next connected file.
4. Use Ctrl+F for the method/expression if your line numbers have changed.

The numbered excerpts are reading aids. Do not paste them into PHP because the line numbers are not valid source code. Never show the private .env, reset URL, cookies, or SMTP password while answering.

This standalone guide includes the full questions, explained answers, short spoken answers, and the source code to show for each answer.

## Quick question-to-file index

| Question | First file to open | First code to point to |
|---|---|---|
| 1 | composer.json | Line 15: Installed Breeze dependency |
| 2 | app/Http/Controllers/Auth/RegisteredUserController.php | Line 41: Registration validates account details |
| 3 | routes/web.php | Line 42: Product routes: auth applied to the whole group |
| 4 | routes/web.php | Line 42: All product display/write routes protected |
| 5 | config/mail.php | Line 9: Default mailer selected from MAIL_MAILER |
| 6 | config/mail.php | Line 21: Private SMTP username/password are read, not hard-coded |
| 7 | app/Http/Requests/Auth/LoginRequest.php | Line 56: Authentication: credential check |
| 8 | routes/web.php | Line 42: Product group uses auth only |

## Question 1: What is Laravel Breeze, and what problem does it solve?

### Answer explained

Breeze is a Laravel authentication starter package that generates routes, controllers, request validation, views, and tests for common account features. It provides registration, login, logout, password recovery, and profile-related code, so the developer can inspect/customize these flows rather than write every piece from an empty project.

In this application, Composer installed Breeze, then `breeze:install` generated the Blade/Alpine scaffold. npm installed frontend tools and the build compiled their assets. Breeze did not automatically protect our existing Product routes; we added the auth middleware group separately.

### Where the answer appears in the code

Show the installed package first, then one generated route/controller/view connection. Breeze supplies account scaffolding; the package declaration alone does not prove every feature has been configured.

Say this while pointing:

"Breeze is installed as a development package. Its auth routes call these controllers and views for registration, login, logout, and recovery. We customized those views and added auth protection to our product routes separately."

### Installed Breeze dependency

File: `composer.json`. Line 15.

```
15:         "laravel/breeze": "^2.4",
```

### Login route selects AuthenticatedSessionController::create

File: `routes/auth.php`. Line 27 through 28.

```
27:     Route::get('login', [AuthenticatedSessionController::class, 'create'])
28:         ->name('login');
```

### Controller returns the login view

File: `app/Http/Controllers/Auth/AuthenticatedSessionController.php`. Line 22 through 25.

```
22:     public function create(): View
23:     {
24:         return view('auth.login');
25:     }
```

### Login form posts to Breeze login route

File: `resources/views/auth/login.blade.php`. Line 23.

```
23:     <form method="POST" action="{{ route('login') }}">
```

## Question 2: What is the difference between registration and authentication?

### Answer explained

Registration creates an account. Authentication verifies that someone using an existing account supplied valid credentials and establishes their signed-in state. Our registration handler also signs in the newly created user, but those remain two different operations in its code.

Example: entering a new email/password on Register creates a users row with a password hash. Later, submitting Login checks that saved account and hash; it does not create another users row. An email already registered is rejected by the unique-email rule on Register, while the same email is expected on Login.

### Where the answer appears in the code

Show account insertion in registration, then credential checking in LoginRequest. They are separate operations even though registration subsequently signs in the new account.

Say this while pointing:

"Registration creates a User record with a hashed password. Authentication checks the saved credentials with Auth::attempt and establishes the signed-in session; it does not insert another user."

### Registration validates account details

File: `app/Http/Controllers/Auth/RegisteredUserController.php`. Line 41 through 46.

```
41:         $request->validate([
42:             'name' => ['required', 'string', 'max:255'],
43:             'email' => ['required', 'string', 'lowercase', 'email', 'max:255',
'unique:'.User::class],
44:             'password' => ['required', 'confirmed', Rules\Password::defaults()],
45:         ]);
46:
```

### Registration inserts User with Hash::make

File: `app/Http/Controllers/Auth/RegisteredUserController.php`. Line 50 through 54.

```
50:         $user = User::create([
51:             'name' => $request->name,
52:             'email' => $request->email,
53:             'password' => Hash::make($request->password),
54:         ]);
```

### Login checks existing credentials and remember value

File: `app/Http/Requests/Auth/LoginRequest.php`. Line 56.

```
56:         if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember'))) {
```

### Successful login rotates the session ID

File: `app/Http/Controllers/Auth/AuthenticatedSessionController.php`. Line 39.

```
39:         $request->session()->regenerate();
```

## Question 3: What is the purpose of the auth middleware?

### Answer explained

auth middleware checks the session's authentication before allowing the request to reach its protected controller. For our HTML product routes, a guest is redirected to named login. A valid signed-in session lets the request continue.

The group in web.php contains all six product routes, including POST store, PUT update, and DELETE destroy. Protecting only the list page would leave those other actions accessible through direct requests.

### Where the answer appears in the code

Point to the product group, not the earlier profile group. The product group is the second auth group in web.php. Its check runs before the controller.

Say this while pointing:

"This auth middleware group requires login for all six Product routes. A guest request is redirected to the named login route before ProductController runs."

### Product routes: auth applied to the whole group

File: `routes/web.php`. Line 42 through 60.

```
42: Route::middleware('auth')->group(function () {
43:     Route::get('/product', [ProductController::class, 'index'])
44:         ->name('product.index');
45:
46:     Route::get('/product/create', [ProductController::class, 'create'])
47:         ->name('product.create');
48:
49:     Route::post('/product', [ProductController::class, 'store'])
50:         ->name('product.store');
51:
52:     Route::get('/product/{product}/edit', [ProductController::class, 'edit'])
53:         ->name('product.edit');
54:
55:     Route::put('/product/{product}', [ProductController::class, 'update'])
56:         ->name('product.update');
57:
58:     Route::delete('/product/{product}', [ProductController::class, 'destroy'])
59:         ->name('product.destroy');
60: });
```

### Named login destination used by the guest redirect

File: `routes/auth.php`. Line 27 through 28.

```
27:     Route::get('login', [AuthenticatedSessionController::class, 'create'])
28:         ->name('login');
```

## Question 4: Why should Product CRUD routes be protected from unauthenticated users?

### Answer explained

Product records are application data. If visitors could call store/update/destroy without authenticating, they could create or change records without an account. Protecting every action establishes the lab's login boundary, including direct requests that do not use the visible buttons.

This boundary does not decide ownership or administrator privileges. The current Product model has no user_id, and there are no policies checking who owns a product. All authenticated users can still manage the shared list.

### Where the answer appears in the code

Show that read and write routes share the same protection. Then point to one write method to explain the database action that a guest must not reach.

Say this while pointing:

"Guests must not directly read or change these product records. The group protects store, update, and destroy as well as the list/forms. Hiding buttons alone would not prevent a direct request."

### All product display/write routes protected

File: `routes/web.php`. Line 42 through 60.

```
42: Route::middleware('auth')->group(function () {
43:     Route::get('/product', [ProductController::class, 'index'])
44:         ->name('product.index');
45:
46:     Route::get('/product/create', [ProductController::class, 'create'])
47:         ->name('product.create');
48:
49:     Route::post('/product', [ProductController::class, 'store'])
50:         ->name('product.store');
51:
52:     Route::get('/product/{product}/edit', [ProductController::class, 'edit'])
53:         ->name('product.edit');
54:
55:     Route::put('/product/{product}', [ProductController::class, 'update'])
56:         ->name('product.update');
57:
58:     Route::delete('/product/{product}', [ProductController::class, 'destroy'])
59:         ->name('product.destroy');
60: });
```

### Insert after validation

File: `app/Http/Controllers/ProductController.php`. Line 65.

```
65:         Product::create($data);
```

### Persist changes to the selected product

File: `app/Http/Controllers/ProductController.php`. Line 100.

```
100:         $product->update($data);
```

### Permanently delete the selected product

File: `app/Http/Controllers/ProductController.php`. Line 112.

```
112:         $product->delete();
```

## Question 5: What is the role of SMTP configuration in password recovery?

### Answer explained

SMTP configuration tells the mailer which email server to contact, on which port, and which credentials/sender identity to use. Laravel's password broker creates the reset request and notification; the SMTP transport delivers that email. SMTP itself does not validate the reset token or save the new password.

We tested Mailtrap's sandbox and Gmail one at a time. Mailtrap captured the email in a test inbox; Gmail delivered it to the registered account's inbox. The email link opened the form with a token, which NewPasswordController passed to the broker for verification before updating the account password.

### Where the answer appears in the code

Show the config keys read from the environment, then the broker calls for sending versus resetting. Never display the real .env password during the demonstration.

Say this while pointing:

"The SMTP config tells the mailer how to deliver the reset notification. sendResetLink asks the password broker to create/send a reset request. Later reset verifies the submitted token before the password-changing callback runs."

### Default mailer selected from MAIL_MAILER

File: `config/mail.php`. Line 9.

```
9:     'default' => env('MAIL_MAILER', 'log'),
```

### SMTP transport reads environment host/port/credentials

File: `config/mail.php`. Line 15 through 24.

```
15:         'smtp' => [
16:             'transport' => 'smtp',
17:             'scheme' => env('MAIL_SCHEME'),
18:             'url' => env('MAIL_URL'),
19:             'host' => env('MAIL_HOST', '127.0.0.1'),
20:             'port' => env('MAIL_PORT', 2525),
21:             'username' => env('MAIL_USERNAME'),
22:             'password' => env('MAIL_PASSWORD'),
23:             'timeout' => null,
24:             'local_domain' => env('MAIL_EHLO_DOMAIN', parse_url((string) env('APP_URL',
'http://localhost'), PHP_URL_HOST)),
```

### Request reset token/email

File: `app/Http/Controllers/Auth/PasswordResetLinkController.php`. Line 41 through 43.

```
41:         $status = Password::sendResetLink(
42:             $request->only('email')
43:         );
```

## Verify reset request, then callback changes password

File: `app/Http/Controllers/Auth/NewPasswordController.php`. Line 49 through 61.

```
49:         $status = Password::reset(
50:             $request->only('email', 'password', 'password_confirmation', 'token'),
51:             // This closure runs after token verification. use($request) makes the outer Request
52:             // available inside it. $user is the account selected by the broker.
53:             function (User $user) use ($request) {
54:                 // Set only the explicitly chosen password hash and a new random remember token.
55:                 // forceFill bypasses fillable, so it is limited to these server-selected keys,
56:                 // not arbitrary form input. save() persists the existing User.
57:                 $user->forceFill([
58:                     'password' => Hash::make($request->password),
59:                     'remember_token' => Str::random(60),
60:                 ])->save();
61:
```

## Question 6: Why should email credentials be kept in the .env file?

### Answer explained

They vary by developer/deployment and should not be hard-coded into a controller or committed to a public repository. The private `.env` supplies credentials to `config/mail.php`; a safe `.env.example` can show field names without secrets. The repository ignore rules exclude the real environment file.

An `.env` is not magic encryption. A leaked file or printed configuration still exposes its values. Do not include it in screenshots, paste it into chats, or serve the project root as a public web directory. Application keys and database passwords need the same care. If a credential is published, remove/revoke/rotate it through its provider; deleting it from the current Git file does not erase history.

### Where the answer appears in the code

Use config/mail.php to show env() reads and .gitignore to show exclusion. If the professor needs to see environment keys, use the safe .env.example or redact values; do not project the private file.

Say this while pointing:

"Mail credentials are deployment values read through env(). The real .env is ignored by Git, while the example is allowed. This keeps secrets separate from controller/source code; .env itself is not encryption."

### Private SMTP username/password are read, not hard-coded

File: `config/mail.php`. Line 21 through 22.

```
21:             'username' => env('MAIL_USERNAME'),
22:             'password' => env('MAIL_PASSWORD'),
```

### Root ignore rules exclude env files and allow only the example

File: `workspace root .gitignore`. Line 9 through 10.

```
9: **/.env*
10: !**/.env.example
```

## Question 7: What is the difference between authentication and authorization?

### Answer explained

Authentication verifies identity or signed-in state. Authorization decides whether that identified user is allowed to do a particular action. A user may be authenticated yet lack permission to delete someone else's record.

Example: both a clerk and manager may log in successfully. A separate policy could allow only the manager to delete products. That clerk/manager rule is an example of a possible extension; our code currently has neither role nor policy.

### Where the answer appears in the code

Show Auth::attempt and the auth-only product group. Authorization is a missing additional decision here; do not pretend a policy file exists when it does not.

Say this while pointing:

"Auth::attempt verifies credentials, and this group requires the resulting session. Authorization would additionally decide whether this user may update or delete a particular product. We have no product owner/admin policy yet, so authenticated users share the list."

### Authentication: credential check

File: `app/Http/Requests/Auth/LoginRequest.php`. Line 56.

```
56:         if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember'))) {
```

### Login boundary, not an administrator policy

File: `routes/web.php`. Line 42 through 43.

```
42: Route::middleware('auth')->group(function () {
43:     Route::get('/product', [ProductController::class, 'index'])
```

### All products read without an owner filter

File: `app/Http/Controllers/ProductController.php`. Line 26.

```
26:         $products = Product::all();
```

## Question 8: Does placing a route inside the auth middleware group automatically make a user an administrator? Explain your answer.

### Answer explained

No. The group checks that the visitor is authenticated using the configured guard. It does not add an administrator flag, assign a role, or compare the user against a permission policy. Any signed-in user can reach our Product routes.

To make certain actions administrator-only, the application would need a real role/permission representation and a server-side authorization check, tested with users who are allowed and users who are denied. Merely naming a route admin or making its button invisible does not implement this check.

### Where the answer appears in the code

Show auth-only protection and the actual User model fields. The code does not assign an admin role or check product permissions. A fillable list is assignment configuration, not a permission system.

Say this while pointing:

"No. auth only checks signed-in state. This group contains no role or policy check, and registration does not assign an administrator role. Admin permissions would need separate data and server-side authorization rules."

### Product group uses auth only

File: `routes/web.php`. Line 42 through 43.

```
42: Route::middleware('auth')->group(function () {
43:     Route::get('/product', [ProductController::class, 'index'])
```

### User inherits authentication behavior

File: `app/Models/User.php`. Line 16 through 19.

```
16: class User extends Authenticatable
17: {
18:     /** @use HasFactory<UserFactory> */
19:     use HasFactory, Notifiable;
```

### User mass-assignable fields, not admin permissions

File: `app/Models/User.php`. Line 26 through 30.

```
26:     protected $fillable = [
27:         'name',
28:         'email',
29:         'password',
30:     ];
```

### Registration saves name/email/password, no role

File: `app/Http/Controllers/Auth/RegisteredUserController.php`. Line 50 through 54.

```
50:         $user = User::create([
51:             'name' => $request->name,
52:             'email' => $request->email,
53:             'password' => Hash::make($request->password),
54:         ]);
```

## Common follow-up questions

These are practice questions in addition to the PDF's eight questions. Use the same file/start-line/search-anchor approach.

| Follow-up question | File and start line | Explanation / expression |
|---|---|---|
| Where is login redirected to Products? | app/Http/Controllers/Auth/AuthenticatedSessionController.php, line 43 | Login fallback is Products; intended preserves the original destination |
| Where is logout and the Login redirect? | app/Http/Controllers/Auth/AuthenticatedSessionController.php, line 53 | Logout, clear session, rotate CSRF token, redirect |
| Where does the root URL choose Login or Products? | routes/web.php, line 20 | Home checks auth state before selecting redirect |
| Where is Remember me handled by the server? | app/Http/Requests/Auth/LoginRequest.php, line 56 | boolean(remember) is passed to the guard |
| Where is registration password hashing? | app/Http/Controllers/Auth/RegisteredUserController.php, line 53 | Hash the password before account storage |
| Where is the reset token submitted? | resources/views/auth/reset-password.blade.php, line 19 | Hidden token supplied from the reset URL |
| Where is the reset cooldown and expiry? | config/auth.php, line 40 | 60 minutes expiry; 60 seconds throttle |
| Where is CSRF in the Login form? | resources/views/auth/login.blade.php, line 28 | Hidden form token generated by Blade |
| Where is CSRF in the Add Product form? | resources/views/products/create.blade.php, line 59 | Product POST form protection |
| Where are product input rules? | app/Http/Controllers/ProductController.php, line 49 | store rules; update has the same rules |
| Where are permitted product assignment fields? | app/Models/Product.php, line 24 | Mass-assignment keys, separate from value validation |
| Where do product success messages disappear? | public/js/product-messages.js, line 13 | Browser removes banner after 5000 milliseconds |
| Where are password Show/Hide controls? | public/js/password-visibility.js, line 25 | Browser switches field presentation, not hashing |
| Where is the product Logout form? | resources/views/layouts/product.blade.php, line 87 | POST plus CSRF calls Breeze logout |

## Prepare your own short explanation

For each PDF question, practice answering in one or two sentences, then open the first file in the index and explain the marked expression without reading the sample aloud. Follow it to a second file when necessary: route to controller, controller to model, broker to mail configuration, or form to receiving route.

For authorization/admin questions, explaining that the rule is absent is the accurate answer. Do not point to a theme button, fillable, CSRF, or an auth group as evidence of administrator permissions.